Cyberscope

A TAC Security Company

Audit Report

Robin Flow

July 2026

Files RobinFlowStreamsV2.sol, RobinFlowFeeCollectorV2.sol

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	15 Jul 2026 https://github.com/cyberscope-io/audits/blob/main/2-flow/v1/audit.pdf
Corrected Phase 2	20 Jul 2026

Source Files

Filename	SHA256
RobinFlowStreamsV2.sol	12859af7d22dbac9e13be64a5c3864e16315231586953c2d27d9da8f65737cf6
RobinFlowFeeCollectorV2.sol	47d0186585ea307fe810843a34ab2788341d5601f5d216ae9af2926fd9be3c4a

Overview

The RobinFlow system is a two contract suite built around a token vesting and locking product. Users create streams that release a deposited ERC20 token to a recipient over time, with an optional cliff and a linear or tranched unlock schedule. Cancelable streams return the unvested balance to the sender, and transferable streams can be handed to a new recipient. A flat native coin fee is charged on every stream created. RobinFlowStreams.sol is the vesting core, and RobinFlowFeeCollector.sol is the inherited base that handles the fee logic.

RobinFlowStreams

Holds the full vesting lifecycle creating streams (single or batch), tracking vesting per stream, letting recipients withdraw what's unlocked, and handling cancels and recipient transfers. The contract itself escrows all deposited tokens.

RobinFlowFeeCollector

The shared fee layer every stream creation runs through. It charges a flat native-coin fee, capped by a hard maximum, and lets the owner update the fee or withdraw what's collected. It holds no ERC-20 user funds and has no power over streams.

Findings Breakdown

Critical	0
Medium	0
Minor / Informative	5

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	4	1	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Unresolved
●	MC	Missing Check	Acknowledged
●	RGGE	Recipient Getter Gas Exhaustion	Unresolved
●	UTRD	Unsafe Token Return Decoding	Unresolved
●	VABC	Vesting Accrues Before Cliff	Unresolved

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	RobinFlowFeeCollectorV2.sol#L43,74,87
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function setFlatFee(uint256 newFee) external onlyOwner {...}
function withdrawFees(address to) external onlyOwner {...}
function renounceOwnership() public virtual onlyOwner{...}
function sweep(address to) external onlyOwner {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

MC - Missing Check

Criticality	Minor / Informative
Location	RobinFlowStreamsV2.sol#L145,150,151,152,153,155,199
Status	Acknowledged

Description

V2 prevents backdated streams, requires `end > start`, constrains `cliff` to the interval between `start` and `end`, and prevents `period` from exceeding the stream duration. It does not, however, enforce minimum separation between `start`, `cliff`, and `end`. A stream may therefore start immediately, set `cliff == start` or `cliff == end`, and complete after only one second. The contract also accepts `period` values of `0` and `1` as per-second vesting. This is valid if explicitly intended, but it remains a configuration ambiguity if integrations expect `period` to represent a nonzero tranche interval. The issue is therefore dependent on the intended product-level schedule constraints rather than an arithmetic failure.

```
p.start < block.timestamp ||
p.end <= p.start ||
p.cliff < p.start ||
p.cliff > p.end ||
uint256(p.period) > uint256(p.end) - uint256(p.start)

uint32 per = s.period;
if (per > 1) elapsed = (elapsed / per) * per;
```

Recommendation

Consider defining the permitted schedule shapes explicitly and enforcing corresponding minimum durations, cliff delays, and period ranges. If zero and one represent linear per-second vesting, that behavior should be clearly distinguished from periodic vesting so callers and integrations cannot interpret the same parameters differently.

Team Update

The team has acknowledged that this is not a security issue and states:

The audit also mentions enforcing a minimum spacing between start/cliff/end. That is a product decision (it would forbid legitimate short streams) and is intentionally left to the front-end / caller rather than hard-coded, so genuine short locks remain possible.

RGGE - Recipient Getter Gas Exhaustion

Criticality	Minor / Informative
Location	RobinFlowStreamsV2.sol#L130,167,253,277,289
Status	Unresolved

Description

Every created or transferred stream is appended to the recipient's permanent `_incoming` history. An attacker can create repeated batches of streams naming a victim as recipient, configure them as cancelable with a future cliff, and cancel them before vesting to recover the deposited tokens. Cancellation does not remove the entries, so the victim's history remains permanently enlarged. `incomingStreams` scans this history twice and compares every entry against all preceding entries, producing `N(N-1)` comparisons. Creating 1,000 entries requires five maximum-sized batches and causes approximately 999,000 comparisons. The attacker pays `1,000 × flatFee` plus creation and cancellation gas but can recover the token principal; at a 0.02 ETH fee this costs 20 ETH plus gas, while a zero or low fee makes the attack substantially cheaper. Organic usage can also eventually create the same denial of service.

```
for (uint256 i = 0; i < len; i++) {
    uint256 id = ids[i];
    if (streams[id].recipient != recipient) continue;

    bool seen = false;
    for (uint256 j = 0; j < i; j++) {
        if (ids[j] == id) {
            seen = true;
            break;
        }
    }

    if (!seen) n++;
}
```

Recommendation

Consider maintaining current recipient membership when streams are created, transferred, canceled, or completed, using an indexed collection that supports constant-time addition and removal. Recipient enumeration should also be paginated with a bounded page size so its execution cost cannot grow with the recipient’s complete lifetime history.

UTRD - Unsafe Token Return Decoding

Criticality	Minor / Informative
Location	RobinFlowStreamsV2.sol#L321
Status	Unresolved

Description

The best-effort recipient payment decodes every nonempty token response as an ABI-encoded Boolean. If a token returns fewer than 32 bytes or a noncanonical Boolean, `abi.decode` reverts instead of returning `false`. This rolls back the entire cancellation, including the sender refund executed earlier in the transaction. Although this behavior is uncommon among established tokens, the contract accepts arbitrary token addresses. OpenZeppelin's `SafeERC20` already handles reverting, Boolean-returning, no-return, and malformed-return tokens without directly decoding untrusted response data.

```
function _tryTransfer(address token, address to, uint256 amount) private
returns (bool) {
    (bool success, bytes memory ret) = token.call(
        abi.encodeWithSelector(IERC20.transfer.selector, to, amount)
    );
    return success && (ret.length == 0 || abi.decode(ret, (bool)));
}
```

Recommendation

Consider replacing the custom low-level transfer implementation with OpenZeppelin's `SafeERC20`. Because cancellation must continue when the recipient cannot receive tokens, the non-reverting `trySafeTransfer` variant may be more appropriate than calling `safeTransfer` directly. Transfer failures should be converted into deferred claims without allowing malformed return data to revert the cancellation.

VABC - Vesting Accrues Before Cliff

Criticality	Minor / Informative
Location	RobinFlowStreamsV2.sol#L199
Status	Unresolved

Description

In the `vestedAmount` he linear portion is calculated using elapsed time from `start`. As a result, the linear portion accrues during the interval between `start` and `cliff`, even though it cannot be claimed during that period. At `cliff`, the recipient receives both the complete `cliffAmount` and all linear vesting accumulated between `start` and `cliff`.

```
function vestedAmount(uint256 streamId, uint40 t) public view returns
(uint256) {
    ...
    uint256 linearPortion = uint256(s.deposited) - s.cliffAmount;
    uint256 elapsed = uint256(t) - s.start;
    uint32 per = s.period;
    if (per > 1) elapsed = (elapsed / per) * per;
    uint256 linear = (linearPortion * elapsed) / (uint256(s.end) -
s.start);
    return uint256(s.cliffAmount) + linear;
}
```

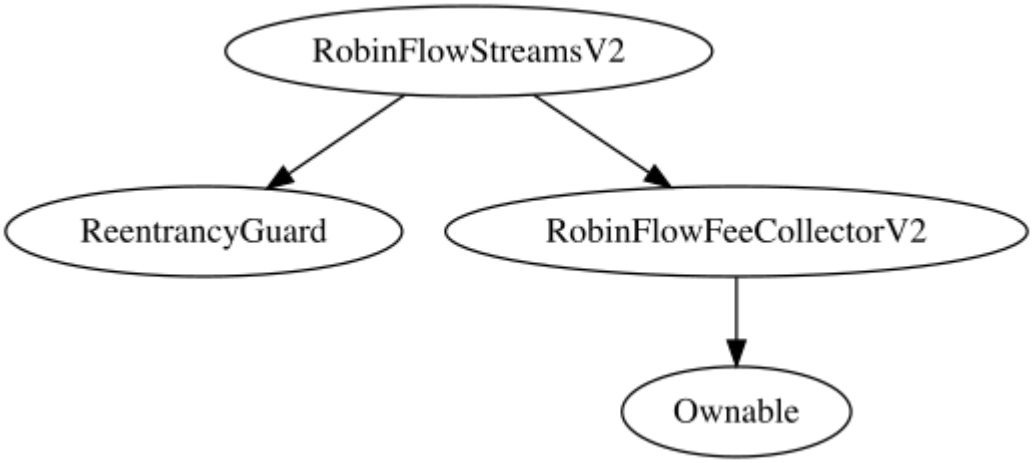
Recommendation

The team is advised to adjust the calculations so that the linear portion begins vesting at cliff, as well as to calculate both elapsed time and duration relative to cliff.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
RobinFlowStreamsV2	Implementation	ReentrancyGuard, RobinFlowFeeCollectorV2		
		Public	✓	RobinFlowFeeCollectorV2
	createStream	External	Payable	nonReentrant
	createStreamsBatch	External	Payable	nonReentrant
	_createStream	Private	✓	
	vestedAmount	Public		-
	withdrawable	Public		-
	withdraw	External	✓	nonReentrant
	cancel	External	✓	nonReentrant
	claimUnclaimed	External	✓	nonReentrant
	transferRecipient	External	✓	-
	outgoingStreams	External		-
	incomingStreams	External		-
	getStream	External		-
	_tryTransfer	Private	✓	

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
RobinFlowFeeCollectorV2
<Constructor>
setFlatFee
_collectFee
RobinFlowFeeCollectorV2.FeeTooHigh
_collectFeeFor
withdrawFees
sweep
RobinFlowFeeCollectorV2.InsufficientFee
RobinFlowFeeCollectorV2.TransferFailed
payable
RobinFlowFeeCollectorV2.NoFees
RobinFlowFeeCollectorV2.ZeroAddress
address
RobinFlowStreamsV2
<Constructor>
createStream
createStreamsBatch
outgoingStreams
incomingStreams
getStream
_collectFee
_createStream
_collectFeeFor
RobinFlowStreamsV2.BatchTooLarge
transferRecipient
claimUnclaimed
withdraw
cancel
type
address
RobinFlowStreamsV2.InvalidParams
RobinFlowStreamsV2.NotTransferable
IERC20
RobinFlowStreamsV2.NotRecipient
RobinFlowStreamsV2.AlreadyCanceled
RobinFlowStreamsV2.NothingToWithdraw
withdrawable
vestedAmount
RobinFlowStreamsV2.NotSender
_tryTransfer
RobinFlowStreamsV2.NotCancelable
token
call

Summary

RobinFlow contracts implement a token vesting and locking product built around cliffs, linear or tranched release schedules, and cancelable or transferable streams, funded through a flat native coin creation fee. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io